

Chapter 1

Introduction to Theory of Computation

1.1 Introduction

- Introduce myself – name, background, research interests, contact information
- Course syllabus
 1. Homework assignments (15%): About 4 assignments throughout the semester. One week to solve the problems. Due at the beginning of class on due date. Late submissions will be penalized.
 2. Quizzes (20%): About 4 quizzes. May be unannounced.
 3. Mid semester exam (20%): 14th September, 2015. 7:30 am – 9:30 am. You may consult your course textbook and class notes in the exam.
 4. End semester exam (45%): 16th November, 2015. 9 am – 12 noon. You may consult your course textbook and class notes in the exam.
- Course textbook: Introduction to the Theory of Computation by Michael Sipser. International student edition.
- Course website: <http://moodle.cse.iitk.ac.in>
- TAs: Information will be posted on moodle soon.

1.2 What is Theory of Computation?

- We first need to understand what is computation. Step by step solution of a problem. Example: multiplication, searching for a word in a dictionary, solving a crossword puzzle, (browsing the web?).
- Computational devices: calculator, computer, phone, pen and paper, etc. There is a vast set of computational. Introduce mathematical models that characterize all such computational devices based on their fundamental property and study them. We will be studying power and limitations of these models.
- How do we come up with a solution? (Algorithms)

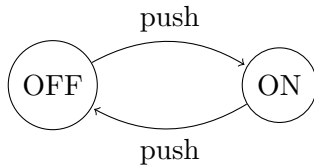
- What can be computed? Are there problems that cannot be computed? We will answer this question later in the semester.

1.3 Introduction to Finite Automata

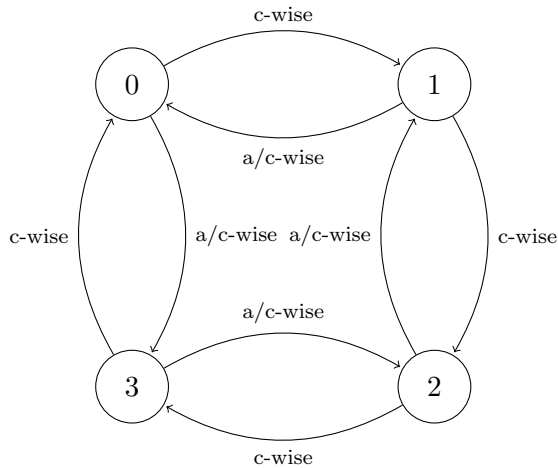
Informally, a finite automaton (plural is automata) is a system consisting of a set of states and interaction between them.

Examples,

1. A switch

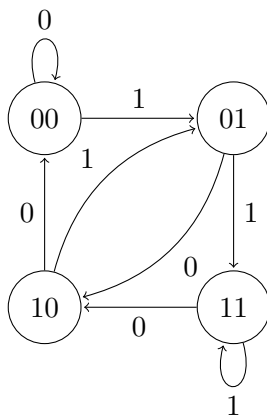


2. A fan regulator



3. Traffic lights

4. Checking if a binary number is divisible by 4. We will use the fact that a binary number is divisible by 4 if and only if the last two digits are 0.



Exercise 1. Read Chapter 0 from textbook.

Let us look at one more example of a finite state machine.

1.4 Another Example

Assume that in a chemistry lab there are 3 compounds, A , B and C , and they react as follows:



Properties:

- The reactions are symmetric
- Total no. of units are the same

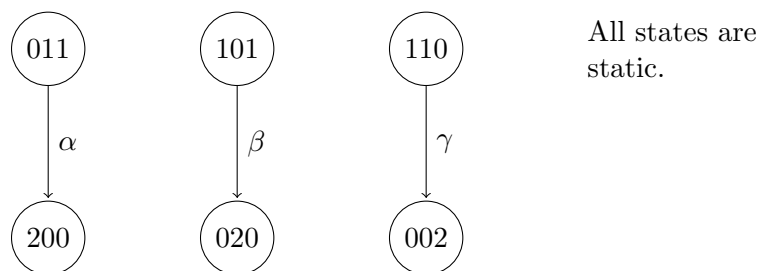
We say the lab is *dysfunctional* if only 1 compound is left.

At any given time the state of the lab is

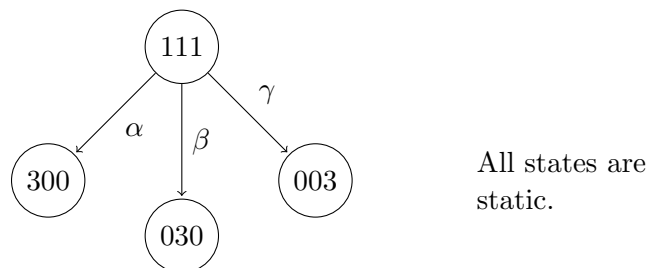
- *static*, if it will/has become dysfunctional. Eg. only one compound is left, or, 1 unit of A and 1 unit of B is left.
- *intermediate*, if it can become dysfunctional. Eg. 2 units of A , 1 units of B , 1 units of C .
- *dynamic*, if it cannot become dysfunctional. Eg. 2 units of A and 1 unit of B .

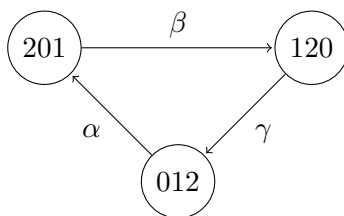
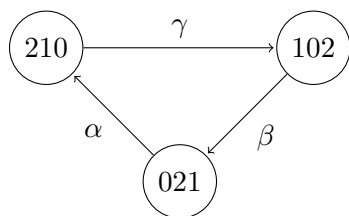
We define a *state* to be a sequence of 3 integers denoting the no. of units A , B and C resp.

Case: 2 units in the lab



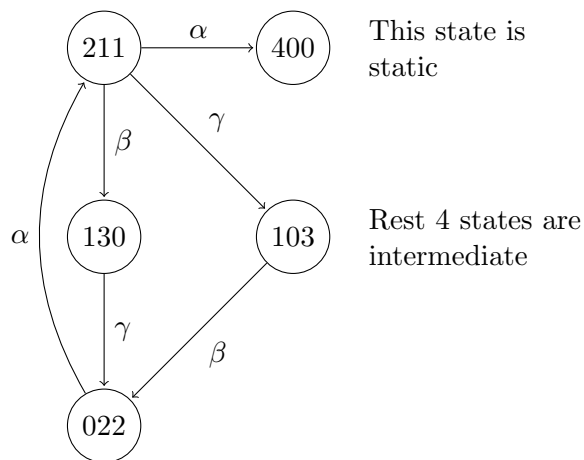
Case: 3 units in the lab





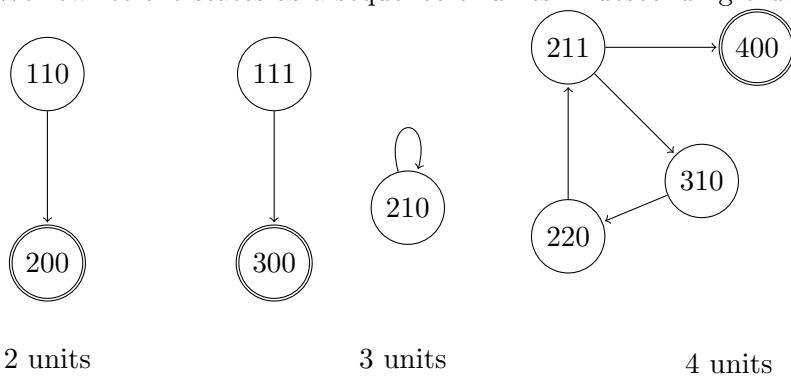
All states are dynamic.

Case: 4 units in the lab

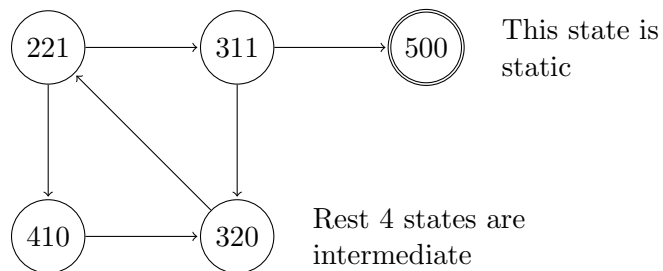


Note: The lab becoming dysfunctional is dependent on the “set” of numbers of each compound, and not on the “ordered triplet”. This is because of the symmetry of the three reactions. For example, the state **211** and **121** can be considered the same.

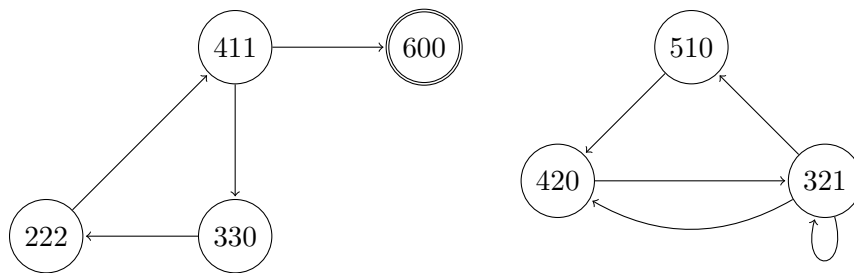
We rewrite the states as a sequence of units in descending order of the numbers.



Case: 5 units in the lab



Case: 6 units in the lab



- State 600 is static.
- States 411, 330, 222 are intermediate.
- States 510, 420, 321 are dynamic.

Exercise 2. 1. Draw the diagram for 7 units.

2. Read chapter 0 from textbook.

We will now setup a mathematical framework to define and study various computational models.

1.5 Basic Definitions

- An *alphabet* is a finite, non-empty set symbols. Convention: Σ, Γ . Egs: $\{0, 1\}$, $\{a, b, \dots, z\}$.
- A *string* is a finite sequence of symbols chosen over some fixed alphabet. Eg. 0110.
- The *empty string* is the string having 0 symbols. Denoted as ϵ .
- *Length of a string* is the number of symbols in it. Eg. $|0110| = 4$, $|\epsilon| = 0$.
- For an alphabet Σ ,

$$\Sigma^i = \{w | w \text{ is a string over } \Sigma \text{ of length } i\}.$$
- $\Sigma^* = \bigcup_{i \geq 0} \Sigma^i$ and $\Sigma^+ = \bigcup_{i > 0} \Sigma^i$
- L is a *language* over Σ if $L \subseteq \Sigma^*$. Note that L can be Σ^* as well.

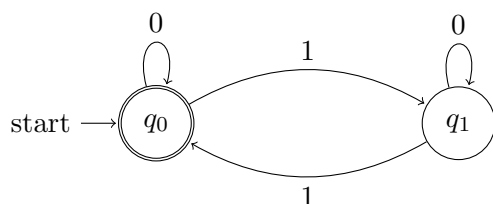
Chapter 2

Deterministic Finite Automata

2.1 Deterministic Finite Automaton

Consider the following language,

$$L = \{x \in \{0, 1\}^* | x \text{ has an even no. of 1's}\}$$



State diagram of an automaton that accepts L

Definition 2.1.1. A *deterministic finite automaton* (in short DFA) is a 5 tuple $(Q, \Sigma, \delta, q_0, F)$, where,

- Q is a finite set called the set of *states*,
- Σ is a finite set called the *alphabet*,
- $\delta : Q \times \Sigma \rightarrow Q$ is the *transition function*,
- $q_0 \in Q$ is the *initial state*, and
- $F \subseteq Q$ is the set of *final states*.

We will now introduce some terminology related to DFA. Let $M = (Q, \Sigma, \delta, q_0, F)$.

- We say M *accepts* $x \in \Sigma^*$ if M halts at an accept state when given x .

-

$$L = L(M) = \{x \in \Sigma^* | M \text{ accepts } x\}$$

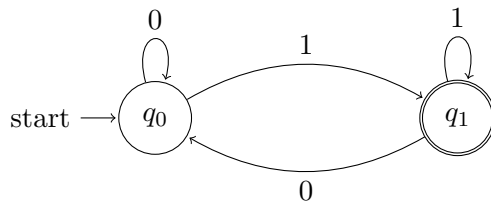
In such a case, we say that M *accepts* L or M *recognizes* L . The terms ‘accept’ and ‘recognize’ will be used synonymously in this course.

Remark. If we say M accepts L it means that M accepts every string in L and every string accepted by M is in L . In other words, both the sets are equal. It is a common rookie mistake to think of one as a proper subset of the other. Please avoid it!

Examples of DFA

1.

$$L_1 = \{x \in \{0, 1\}^* | x \text{ ends with a } 1\}$$

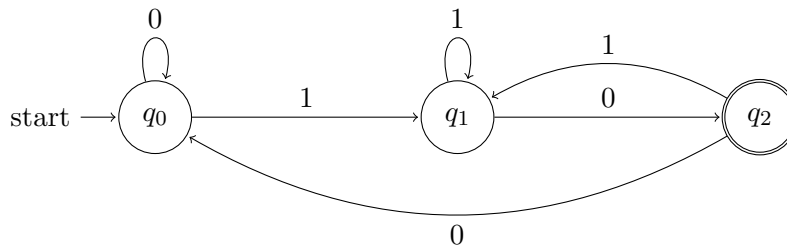


State diagram of an automaton that accepts L_1

Intuition of the various states:

- State q_0 corresponds to all those strings that end with a 0.
- State q_1 corresponds to all those strings that end with a 1.

2. Consider the following DFA



State diagram of DFA M_2

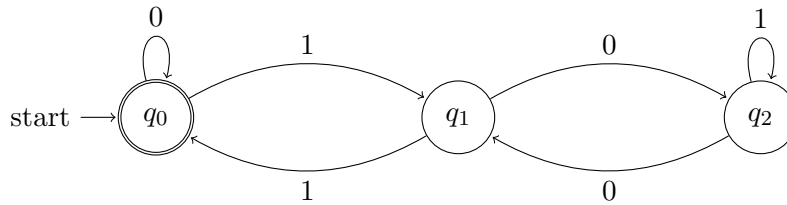
$$L(M_2) = \{x \in \{0, 1\}^* | x \text{ ends with a } 10\}$$

Intuition of the various states:

- State q_0 corresponds to all those strings that end with two 0's or the string 0.
- State q_1 corresponds to all those strings that end with a 1.
- State q_2 corresponds to all those strings that end with a 10.

3.

$$L_3 = \{x \in \{0, 1\}^* | x \text{ is divisible by } 3\}$$



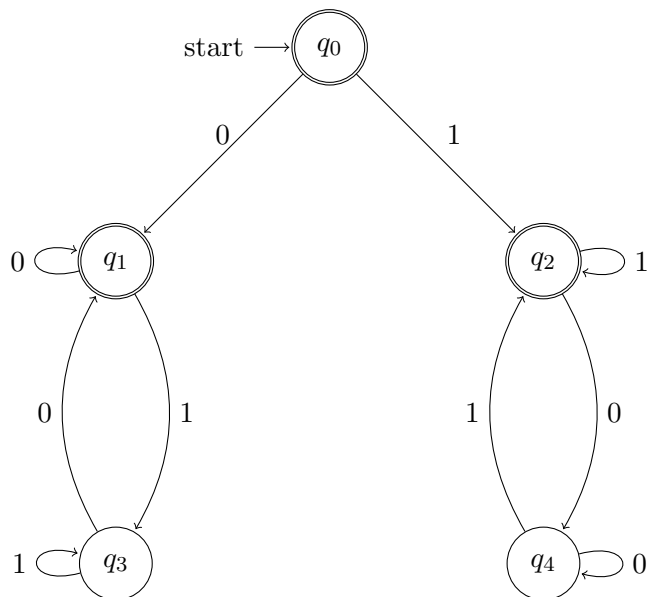
State diagram of DFA M_3 such that $L_3 = L(M_3)$

Intuition of the various states:

- q_0 : all strings x such that $x \equiv 0 \pmod 3$.
- q_1 : all strings x such that $x \equiv 1 \pmod 3$.
- q_2 : all strings x such that $x \equiv 2 \pmod 3$.

4.

$$L_4 = \{x \in \{0,1\}^* \mid x \text{ begins and ends with the same symbol}\}$$



State diagram of DFA M_4 such that $L_4 = L(M_4)$

Intuition of the various states:

- q_0 : accepts ϵ .
- q_1 : all strings beginning and ending with 0.
- q_3 : all strings beginning with 0 and ending with 1.
- q_2 and q_4 are symmetric to q_1 and q_3 .

Note 1. - Given a language there exist many DFAs (infact infinitely many) that accept the given language. However, given a DFA, there is only a unique language that the DFA accepts.

- The size of an automaton is independent of the length of the input string given to it. Strings can be arbitrarily long. This is the meaning of the term *finite* in a DFA.
- An automaton reads one bit at a time.
- At any instant while reading an input, the automaton does not know when the string will terminate. Its action at that point depends only on the (i) current state, and (ii) currently read input symbol.
- For every state p and every input symbol a , there is a unique state q that a DFA will go to from p on seeing the symbol a . This means that once a string is fixed, the automaton has a unique walk starting from the start state. If the last state is an accept state then the string is accepted, otherwise not. This is the meaning of the term *deterministic* in a DFA.
- The states of a DFA informally correspond to some property. Since a DFA has a finite number of states, hence it can “remember” only a finite amount of information of the input string. For example, a DFA cannot count the number of 1's present in a binary input string. We will prove such limitations of a DFA later in this course.

2.2 Definition of Computation of a DFA

Let $M = (Q, \Sigma, \delta, q_0, F)$ be a DFA. Let $w = a_1a_2 \dots a_n$ be a string in Σ^* .

- We say that M *accepts* w if there exists a sequence of states $r_0, r_1, \dots, r_n \in Q$ (not necessarily distinct), such that

1. $r_0 = q_0$,
2. $r_i = \delta(r_{i-1}, a_i)$ for $i = 1, 2, \dots, n$
3. $r_n \in F$.

- We say that M *accepts* $A \subseteq \Sigma^*$ if

$$A = \{w \in \Sigma^* | M \text{ accepts } w\}$$

Denoted as $L(M)$. Once again note that M accepts a language A if and only if M accepts *exactly* those strings that belong to A .

- A language L is said to *regular* if $L = L(M)$ for some DFA M .

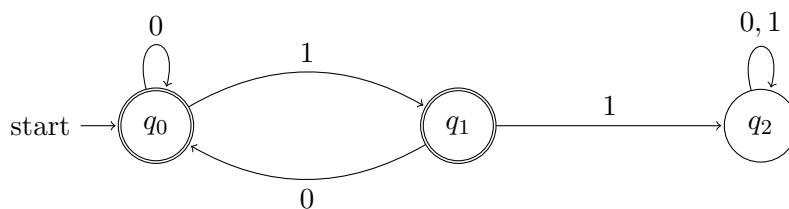
So far we have seen several regular languages. Here is an example of a non-regular language. We will prove this later.

$$L = \{0^n 1^n | n \geq 0\}$$

Some more examples

1.

$$L_1 = \{x \in \{0,1\}^* \mid x \text{ does not contain the substring } 11\}$$



State diagram of DFA M_1 such that $L_1 = L(M_1)$

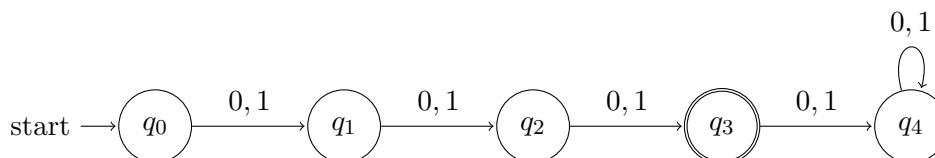
Intuition of the various states:

- q_0 : strings that do not contain 11 and end with a 0.
- q_1 : strings that do not contain 11 and end with a 1.
- q_2 : strings that contain 11.

Remark. States from which the automaton can never reach an accept state are known as *dump states*. The state q_2 in the above DFA is an example of a dump state.

2.

$$L_2 = \{x \in \{0,1\}^* \mid |x| = 3\}$$



DFA for L_2

Intuition of the various states:

- q_0 : string of length 0.
- q_1 : strings of length 1.
- q_2 : strings of length 2.
- q_3 : strings of length 3.
- q_4 : strings of length ≥ 4 .

2.3 Regular Operations

Let us define some operations on languages and study their properties. Let $A, B \subseteq \Sigma^*$ be two languages.

1. Union operation:

$$A \cup B = \{x \mid x \in A \text{ or } x \in B\}$$

2. Concatenation operation:

$$A \cdot B = \{xy \mid x \in A \text{ and } y \in B\}$$

This is also denoted as AB .

3. Star operation:

$$A^* = \{x_1x_2 \dots x_k \mid k \geq 0 \text{ and } x_i \in A\}$$

Example: Let $A = \{1, 2\}$ and $B = \{a, b, c\}$. Then

$$\begin{aligned} A \cup B &= \{1, 2, a, b, c\} \\ AB &= \{1a, 1b, 1c, 2a, 2b, 2c\} \\ A^* &= \{\epsilon, 1, 2, 11, 12, 21, 22, 111, 112, 121, 122, 211, 212, 221, 222, \dots\} \end{aligned}$$

Note that AB need not be the same as BA .

2.3.1 Closure Properties

We say that a class of objects is *closed* under some operation if applying that operation to objects of the class, produces an object that belongs to the same class as well.

Example: $\mathbb{N}$ is closed under addition and multiplication but not closed under subtraction.

Theorem 1. *Let A, B be two regular languages. Then $A \cup B$, AB and A^* are also regular. In other words, regular languages are closed under the union, concatenation and star operations.*

We will see a proof of this theorem later in the course.

Exercise 3. Construct DFAs for the following languages.

1. $L_1 = \{w \in \{0, 1\}^* \mid \#_0(w) \text{ is even and } \#_1(w) \text{ is odd}\}$
2. $L_2 = \{w \in \{0\}^* \mid |w| \text{ is divisible by 2 or 7}\}$
3. $L_3 = \{w \in \{0, 1\}^* \mid |w| \text{ is divisible by 5}\}$

Remark. $\#_0(w)$ denotes the number of occurrences of 0 in w . Similarly $\#_1(w)$.

Exercise 4. Read chapter 1.1 from textbook.

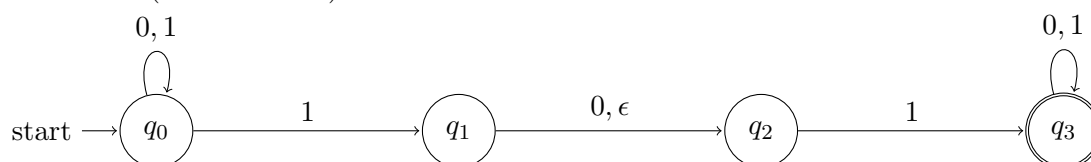
Chapter 3

Nondeterministic Finite Automata

3.1 Nondeterminism

- From a state q on an input symbol a there can be 0 or more transitions to other states (outgoing edges).
- The automaton *simultaneously* moves to all the states in one move.
- Here transitions can also be labelled by ϵ . On an epsilon transition, the automaton moves to the corresponding without reading an input bit.
- The automaton accepts its input if one of the *computation paths* lead to an accept state.

Let us look at an example before moving further. Here is an example of a nondeterministic finite automaton (NFA in short).



- 0, 1 or many transitions from a state on seeing a symbol. Such as
 - $(q_0, 1) \longrightarrow \{q_0, q_1\}$
 - $(q_1, 1) \longrightarrow \{\}$
 - $(q_1, 0) \longrightarrow \{q_2\}$
- Labels on transitions marked with symbols from $\Sigma \cup \{\epsilon\}$. If there is a transition from a state q_i to another state q_j labelled with ϵ , then the automaton simultaneously moves to the states q_i (i.e. stays at the same state) and q_j without reading any input symbol. Example
 - $(q_1, \epsilon) \longrightarrow \{q_1, q_2\}$

3.1.1 Computation of an NFA on a given input

Let us now see how computation happens on an NFA, given an input.

Input: Let the input string be 010110.

Current set of positions of the NFA	Symbol Read
$\{q_0\}$ (Initial position of the NFA)	
$\{q_0\}$	0
$\{q_0\}$	1
$\{q_0, q_1, q_2\}$	0
$\{q_0, q_2\}$	1
$\{q_0, q_1, q_2, q_3\}$	1
$\{q_0, q_1, q_2, q_3\}$	0
$\{q_0, q_2, q_3\}$	Input is accepted since q_3 is an accept state

You may want to picture a nondeterministic computation as follows. From a state q , on an input symbol a , if there are $k(\geq 0)$ outgoing edges then imagine that the automaton “consumes” the bit a , makes k copies of itself at that instant with each copy going to one of the outgoing states. If $k = 0$, then that particular copy of the automaton “dies” on seeing the symbol a . Similarly, from a state q , if there are $k(\geq 0)$ ϵ edges, then the automaton makes $k + 1$ copies of itself at that instant with each copy going to one of the outgoing states and one copy staying at the current state without consuming any input bit.

Computation proceeds in parallel in all the copies. We usually refer to the different copies of the automaton as different *computation paths*.

The input is accepted if the final set of states of the automaton after it has read the input completely, has an accept state.

Remark. Nondeterminism is not a realistic model in the sense that there are no nondeterministic computers in the real world. However often it makes description of the automaton easier.

3.1.2 Difference between deterministic and nondeterministic computations

Deterministic Computation	Nondeterministic Computation
(state, input symbol) pair $\longrightarrow$ A SINGLE state	(state, input symbol) pair $\longrightarrow$ MULTIPLE state (≥ 0) simultaneously
a single computation path	multiple computation paths
computation is at a unique state in each step	computation is in a set of states in each step
input accepted if computation path leads to an accept state	input accepted if at least one computation path leads to an accept state
input rejected if computation path does not lead to an accept state	input rejected if none of the computation paths lead to an accept state
does not allow ϵ transitions	allows ϵ transitions

3.2 Nondeterministic Finite Automaton

Let $\Sigma_\epsilon = \Sigma \cup \{\epsilon\}$. For a set A , let 2^A denote the power set of A .

Definition 3.2.1. A *nondeterministic finite automaton* (in short NFA) is a 5 tuple $N = (Q, \Sigma, \delta, q_0, F)$, where,

- Q is a finite set called the set of *states*,
- Σ is a finite set called the *alphabet*,
- $\delta : Q \times \Sigma_\epsilon \rightarrow 2^Q$ is the *transition function*,
- $q_0 \in Q$ is the *initial state*, and
- $F \subseteq Q$ is the set of *final states*.

We say that N *accepts* $w = a_1a_2 \dots a_n$ if we can write w as $w = b_1b_2 \dots b_m$, where each $b_i \in \Sigma_\epsilon$ and there exists a sequence of states $r_0, r_1, \dots, r_m \in Q$ (not necessarily distinct), such that

1. $r_0 = q_0$,
2. $r_i \in \delta(r_{i-1}, b_i)$ for $i = 1, 2, \dots, m$
3. $r_m \in F$.

Then,

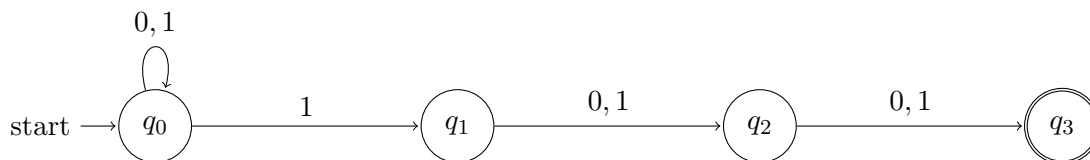
$$L(N) = \{w \in \Sigma^* \mid N \text{ accepts } w\}.$$

3.2.1 Examples of NFA

1. Consider the following language

$$L = \{w \in \{0, 1\}^* \mid \text{the 3rd last symbol of } w \text{ is } 1\}$$

Below is an NFA for the above language.



Exercise 5. Now construct a DFA for the above language. What can you say about the size (i.e. no. of states) of the DFA compared to the NFA? Consider the language

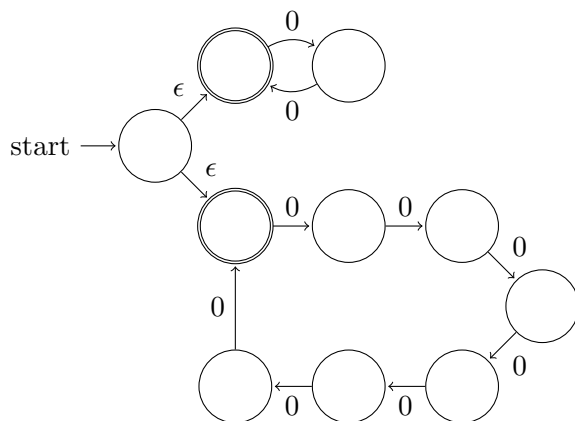
$$L_k = \{w \in \{0, 1\}^* \mid \text{the } k\text{-th last symbol of } w \text{ is } 1\}$$

What is the smallest sized NFA that can accept L_k (as a function of k)? What about the smallest sized DFA?

2. Recall the following language discussed earlier

$$L = \{w \in \{0\}^* \mid |w| \text{ is divisible by 2 or 7}\}$$

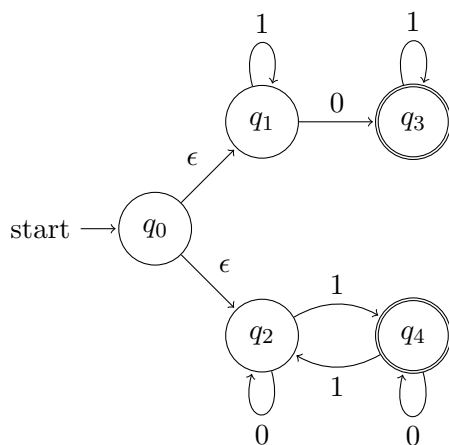
Below is an NFA that accepts L .



How does the size of the DFA and NFA compare with each other for L ?

3. Another example.

$$L = \{w \in \{0\}^* \mid \#_0(w) = 1 \text{ or } \#_1(w) \text{ is odd}\}$$



Exercise 6. Problem 1.5 from chapter 1 in the textbook.

Note 2. Observe that a DFA is also an NFA. It just so happens that a DFA does not use its nondeterminism. Therefore if $\mathcal{N}$ is the class of all languages accepted by some NFA then regular languages are a subset of $\mathcal{N}$. But what about the converse?

Theorem 2. Let $N = (Q, \Sigma, \delta, q_0, F)$ be an NFA. Then there exists a DFA $M = (Q', \Sigma, \delta', q'_0, F')$ such that $L(N) = L(M)$.

We will give the construction of the DFA M . The idea is to consider all subset of states of N .

Construction of M

1. $Q' = 2^Q$.
2. Let R be a state in Q' . Then by definition of Q' , $R \subseteq Q$. We define δ' as follows:

$$\delta'(R, a) = \bigcup_{r \in R} \delta(r, a).$$

3. $q'_o = \{q_0\}$.
4. $F' = \{R \in Q' \mid R \cap F \neq \emptyset\}$.

What about ϵ -transitions?

Let $R \subseteq Q$. Define the ϵ -closure of R as,

$$E(R) = \bigcup_{r \in R} \{q \mid q \text{ can be reached from } r \text{ by using 0 or more } \epsilon\text{-transitions}\}.$$

The transition function δ' can be modified as

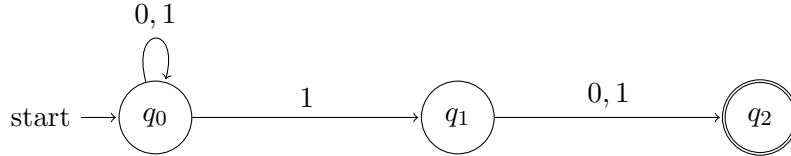
$$\delta'(R, a) = \bigcup_{r \in R} E(\delta(r, a)),$$

and $q'_o = E(\{q_0\})$.

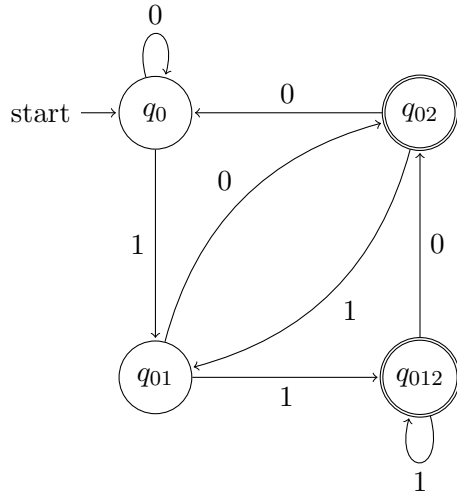
Corollary 3. *A language is regular if and only if it is accepted by some NFA.*

An Example

Consider the language $L = \{w \in \{0, 1\}^* \mid \text{the 2nd last symbol of } w \text{ is } 1\}$. Here is an NFA for L .



Based on the construction given earlier, the corresponding DFA for L will be as follows.



Although there are $2^3 = 8$ subsets of the set $\{q_0, q_1, q_2\}$, but only 4 of them are *reachable* from the start state q_0 . Hence it is sufficient to consider these 4 states.

Remark. States of a DFA/NFA that are not reachable from the start state via any sequence of symbols are redundant and therefore, need not be considered.

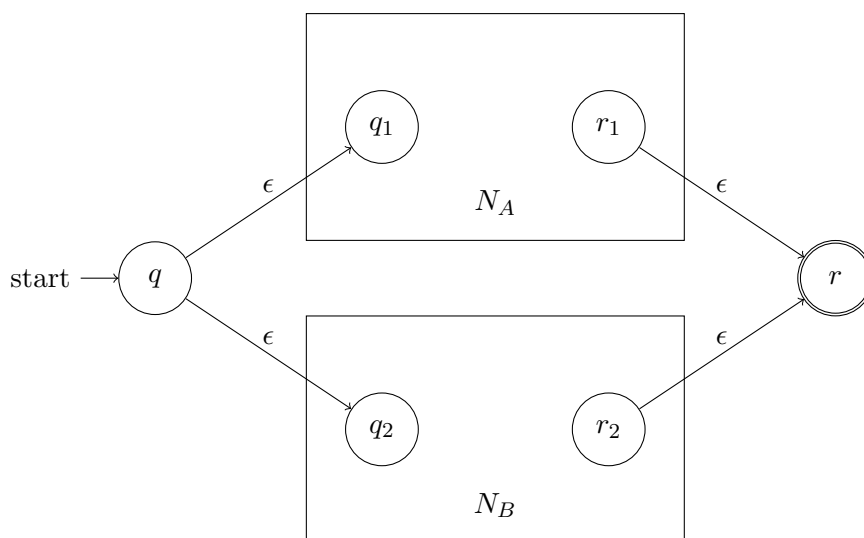
3.3 Closure properties of Regular Languages

Recall the following theorem discussed in the previous Lecture Notes.

Theorem 4. *Let A, B be two regular languages. Then $A \cup B$, AB and A^* are also regular. In other words, regular languages are closed under the union, concatenation and star operations.*

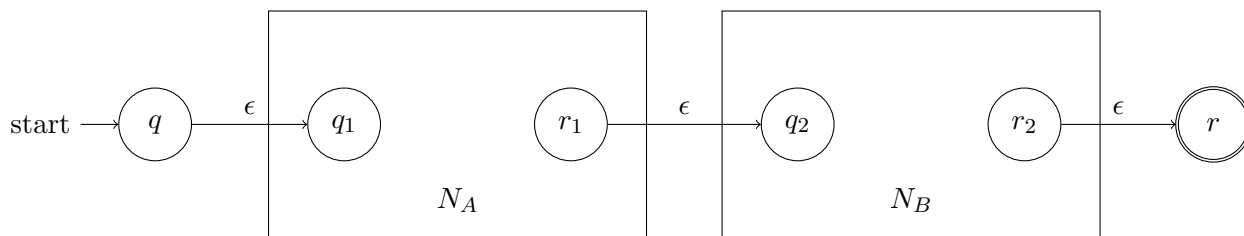
We will give a constructive proof of the theorem. Let N_A and N_B be two NFA's accepting A and B respectively. We assume without loss of generality that N_A and N_B have a unique start state and a unique accept state, with no incoming transitions to the start state and no outgoing edges from the accept state. This is because if the NFA has more than one accept state, then we add a new accept state to the NFA and add ϵ -transitions from the old accept states to the new accept state.

1. Union operation



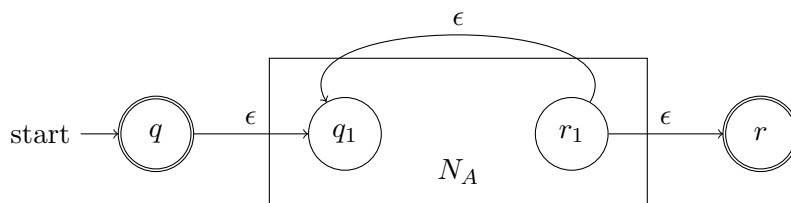
Construction of an NFA that accepts the language $A \cup B$.

2. Concatenation operation



Construction of an NFA that accepts the language AB .

3. Star operation



Construction of an NFA that accepts the language A^* .

Exercise 7. Read Section 1.2 from textbook.